

Module 1 — Core Mastery

Lesson 1.5

Context management

Treat the context window as a budget. Spend it on signal, not noise.

Mastering Claude Code — An E-Course for Power Users

Why it matters

Long sessions accumulate context: file contents Claude read, search results, error tracebacks, your own corrections. When the window fills, quality degrades — Claude forgets what you said earlier, makes inconsistent edits, hallucinates filenames. Context management is the discipline of keeping signal density high.

The mental model

Three levers control context:

- 1 **Compaction** — keep working in the same session but summarize older turns.
- 2 **/clear** — wipe context, keep the session.
- 3 **Subagents as shields** — delegate noisy work to a child agent so the noise never enters your window.

Use the right lever for the situation.

Compaction

Claude Code auto-compacts when context approaches the model's limit. You can also force it: `/compact`.

What compaction does:

- Summarizes older message turns.
- Preserves system context (CLAUDE.md, settings, tool list).
- Keeps recent turns verbatim.

When to force compaction:

- You're deep in a task, want to continue, but notice slow responses or summarization happening implicitly.
- You're about to start a long subroutine and want headroom.

The `PreCompact` hook (covered in 3.3) fires before compaction — useful for snapshotting state you'll want to recall after.

/clear

`/clear` wipes everything except what's loaded at session start (CLAUDE.md, settings, MCP, hooks).

Use when:

- Done with one task, starting a different one. The old context is dead weight.
- A long debugging session has poisoned the context with red herrings.
- You want to test "would Claude do this correctly from a fresh slate?"

Cost: you lose everything you discovered. Mitigation: ask Claude to "save what you learned to a notes file" *before* `/clear`.

Subagents as context shields

The single biggest lever for advanced users. When a task involves reading lots of files or exploring unfamiliar territory:

You ■■■ "Use Explore to find every endpoint that touches billing"



Explore subagent (separate context window)
reads 30 files, runs 12 greps



Returns: "Found 6 endpoints, here are the file:line refs."

Your context grew by ~200 tokens (the report), not 50,000 tokens (everything Explore read).

This is the single most important habit for keeping long sessions usable.

We cover this in depth in Module 4.

What NOT to put in context

- **Large data dumps** — don't paste a 10k-line log. Save it to disk, ask Claude to Grep it.
- **Whole binary files** — never. Reference by path.
- **Generated/minified code** — Claude can't usefully reason over it. Point to the source.
- **"Just in case" file dumps** — only read files you'll actually use.

Try it

- 1 In a session, ask Claude: "How much context have we used so far? Estimate."
- 2 Read a large file (say, a generated `package-lock.json`). Note the visible slowdown.
- 3 `/clear` and try a fresh task — note the snap-back.
- 4 Run the same task again, but this time have Claude delegate file reading to an Explore subagent. Compare felt performance.

Gotchas

- **`/clear` is irrecoverable.** No undo. Save anything important first.
- **Compaction can drop subtle context** — if Claude starts re-asking questions you already answered, that's the symptom. Restate the key constraints.
- **Auto-compaction surprises** — if you're mid-task and behavior shifts, compaction may have just run. The conversation summary message announces it.
- **MCP `resources` and tool outputs count toward context too.** A "read 100 resource URIs" call can spike usage.

Further reading

- Module 4 Subagents — the primary context-management technique for advanced users

- 3.3 Hooks — PreCompact for snapshotting

Next

→ 2.1 The planning loop